镍板缺陷检测：单人全栈重构执行手册

> **目标**：一个人搞定从数据到部署的全部流程 **约束**：不 Docker、Tauri 打开即用、不抛弃旧数据、多光源兼容 **选型**：不再纠结，直接给出唯一可行方案

一、技术选型（已确定）

模块	选型	理由
检测模型	**YOLO11-M**	一个人维护，ultralytics 生态完善，训练/导出/部署一条龙
等级模型	**LightGBM**	可解释，训练快，导出 ONNX 后在 ONNXRuntime 中推理
部署方式	**Tauri + FastAPI Sidecar**	Tauri 启动时自动启动 Python 后端（PyInstaller 打包），关闭时自动结束，零 Docker
推理加速	**ONNX Runtime**	YOLO11 导出 ONNX，用 ONNXRuntime 在 Python 中加载，比 PyTorch 快 2-3 倍

为什么不用纯 Rust 推理?
YOLO11 的后处理（NMS、解码）在 Rust 中重写成本高，一个人搞不定。用 PyInstaller 把 Python 后端打包成 exe，Tauri 作为 sidecar 启动，是最现实的方案。

二、你不懂的概念：用大白话解释

2.1 迁移学习 = 让新模型继承旧知识

场景：YOLO11 在 COCO 数据集（80 类日常物体）上预训练过，但它没见过镍板缺陷。

步骤：

1. **加载预训练权重**：YOLO11-M 的 COCO 权重（已经学会识别边缘、纹理、形状）
2. **冻结 Backbone（主干网络）**：前 20 层不更新参数
   - 为什么？因为这些层学的是通用视觉特征（边缘、角点、纹理），任何图片都有，不需要重学
3. **只训练 Neck（脖子）和 Head（头）**：让模型学会”镍板的缺陷长什么样”
4. **解冻，全量微调**：训练一段时间后，解冻所有层，用很小的学习率再训练几轮

```
from ultralytics import YOLO

# Phase 1: 冻结 Backbone，只训练 Head
model = YOLO('yolo11m.pt')  # COCO 预训练权重
model.train(data='data.yaml', epochs=100, lr0=0.001, freeze=20)

# Phase 2: 解冻全量微调
model = YOLO('runs/train/phase1/weights/last.pt')
model.train(data='data.yaml', epochs=50, lr0=0.0001, freeze=0)
```

2.2 伪彩色合成 = 不改模型，只改输入图片

核心：YOLO11 的输入永远是 3 通道（RGB）。我们不改模型任何结构，只是把多光源图片”伪装”成 RGB 图片。

数据类型	R 通道	G 通道	B 通道
单光历史	单光灰度	单光灰度	单光灰度
双光新	左光	右光	(左 + 右)/2
三光新	左光	右光	融合光

```
import numpy as np

# 单光 -> 复制三通道
pseudo = np.stack([gray, gray, gray], axis=2)

# 三光 -> 左/右/融合分别填入 R/G/B
pseudo = np.stack([left, right, fusion], axis=2)
```

为什么模型能学会?
模型看到的是”红色通道有划痕特征，绿色通道没有，蓝色通道有融合特征”，它自己会把这些通道关联起来。

2.3 ONNX / TensorRT = 让模型跑得更快

- **PyTorch (.pt)**：开发时用的格式，加载慢，推理慢（100ms）
- **ONNX (.onnx)**：通用格式，任何框架都能加载，推理中等（50ms）
- **TensorRT (.engine)**：NVIDIA 专用，推理最快（15ms）

你的部署路径：

```
YOLO11 训练 (PyTorch .pt)
    -> 导出 ONNX (.onnx) ——必须做
    -> 可选 TensorRT (.engine) ——更快，但非必须
```

一个人搞定的现实选择：导出 ONNX，用 ONNX Runtime 推理。速度足够（50ms/张），且不需要 NVIDIA GPU 也能跑。

三、数据准备与处理

3.1 数据集目录结构

```
datasets/
├── dianjienie/
│   ├── images/
│   │   ├── train/
│   │   │   ├── single/          # 单光历史数据
│   │   │   ├── left/            # 左光新数据
│   │   │   ├── right/           # 右光新数据
│   │   │   └── fusion/          # 融合光新数据
│   │   └── val/
│   ├── labels/
│   │   ├── train/               # YOLO 标签（所有光源共用）
│   │   └── val/
```

```
|   └── grade_labels/
|       ├── old_grades.csv       # 旧等级标签
|       └── train_features.csv   # 提取的特征（AI 生成）
```

关键约定：同一块镍板的单光/左光/右光/融合光，文件名必须相同。

3.2 伪彩色预处理脚本

```python
# scripts/preprocess.py
import cv2
import numpy as np
from pathlib import Path

class MultiLightPreprocessor:
    def __init__(self, input_dir, output_dir, target_size=640):
        self.input_dir = Path(input_dir)
        self.output_dir = Path(output_dir)
        self.target_size = target_size
        self.output_dir.mkdir(parents=True, exist_ok=True)

    def process_single(self, img_path):
        img = cv2.imread(str(img_path), cv2.IMREAD_GRAYSCALE)
        if img is None: return None
        img = cv2.resize(img, (self.target_size, self.target_size))
        pseudo = np.stack([img, img, img], axis=2)
        return pseudo

    def process_triple(self, left_path, right_path, fusion_path):
        left = cv2.imread(str(left_path), cv2.IMREAD_GRAYSCALE)
        right = cv2.imread(str(right_path), cv2.IMREAD_GRAYSCALE)
        fusion = cv2.imread(str(fusion_path), cv2.IMREAD_GRAYSCALE)
        if left is None or right is None or fusion is None: return None

        left = cv2.resize(left, (self.target_size, self.target_size))
        right = cv2.resize(right, (self.target_size, self.target_size))
        fusion = cv2.resize(fusion, (self.target_size, self.target_size))
        pseudo = np.stack([left, right, fusion], axis=2)
        return pseudo

    def run(self):
        single_dir = self.input_dir / 'single'
        left_dir = self.input_dir / 'left'
        right_dir = self.input_dir / 'right'
        fusion_dir = self.input_dir / 'fusion'

        # 处理单光
        if single_dir.exists():
            for img_path in single_dir.glob('*.jpg'):
                pseudo = self.process_single(img_path)
                if pseudo is not None:
                    cv2.imwrite(str(self.output_dir / f"{img_path.stem}_s.jpg"), pseudo)
```

```
        # 处理三光
        if left_dir.exists() and right_dir.exists() and fusion_dir.exists():
            for left_path in left_dir.glob('*.jpg'):
                right_path = right_dir / left_path.name
                fusion_path = fusion_dir / left_path.name
                if right_path.exists() and fusion_path.exists():
                    pseudo = self.process_triple(left_path, right_path, fusion_path)
                    if pseudo is not None:
                        cv2.imwrite(str(self.output_dir / f"{left_path.stem}_t.jpg"), pseudo)

        print(f"Done. Output: {self.output_dir}")

if __name__ == '__main__':
    MultiLightPreprocessor(
        input_dir='datasets/dianjienie/images/train',
        output_dir='datasets/dianjienie/images_pseudo/train',
        target_size=640
    ).run()
```

四、检测模型训练（YOLO11-M）

4.1 数据配置

```
# datasets/dianjienie/data.yaml
path: datasets/dianjienie
train: images_pseudo/train
val: images_pseudo/val
nc: 18
names: [wuzi, waterstain, yellowstain, nodule, airhole, bubble, bump, fawu, overfire, NiOH, greenstain,
↪  suanwuran, quejiao, watermark, youpao, heibian, mijixiaokeng, fanxitong]
```

4.2 训练脚本

```
# scripts/train_yolo11.py
from ultralytics import YOLO

def train():
    # Phase 1: 冻结 Backbone
    model = YOLO('yolo11m.pt')
    model.train(
        data='datasets/dianjienie/data.yaml',
        epochs=100, imgsz=640, batch=16,
        lr0=0.001, freeze=20,  # 冻结前 20 层
        patience=30,
        project='runs/dianjienie', name='phase1',
        device=0
    )
```

```
    # Phase 2: 全量微调
    model = YOLO('runs/dianjienie/phase1/weights/last.pt')
    model.train(
        data='datasets/dianjienie/data.yaml',
        epochs=50, imgsz=640, batch=16,
        lr0=0.0001, freeze=0,  # 不冻结
        patience=20,
        project='runs/dianjienie', name='phase2',
        device=0
    )

    # 导出 ONNX
    model = YOLO('runs/dianjienie/phase2/weights/best.pt')
    model.export(format='onnx', imgsz=640, simplify=True)
    print("Export complete: runs/dianjienie/phase2/weights/best.onnx")

if __name__ == '__main__':
    train()
```

五、等级数据集优化 + LightGBM 训练

5.1 旧数据集的问题与优化

问题：旧数据集靠肉眼缩小图片看光泽分等级，光照影响大，没有考虑缺陷。

优化方案：用训练好的 YOLO11 对旧数据跑一遍，提取缺陷统计特征，用这些特征 + 原等级标签训练 LightGBM。

```
# scripts/extract_features.py
import pandas as pd
import numpy as np
from ultralytics import YOLO
import cv2
from pathlib import Path
from tqdm import tqdm

def extract_features(image_dir, model_path, output_csv):
    model = YOLO(model_path)
    records = []

    for img_path in tqdm(list(Path(image_dir).glob('*.jpg'))):
        img = cv2.imread(str(img_path))
        if img is None: continue

        results = model(str(img_path), verbose=False)
        boxes = results[0].boxes

        features = {'image_id': img_path.stem}
        total_area = img.shape[0] * img.shape[1]

        for cls_id in range(18):
```

```
            cls_name =
            ↪ ['wuzi','waterstain','yellowstain','nodule','airhole','bubble','bump','fawu','overfire','NiOH','greenstai
            cls_boxes = [b for b in boxes if int(b.cls[0]) == cls_id]
            features[f'{cls_name}_count'] = len(cls_boxes)
            features[f'{cls_name}_area'] = sum((b.xyxy[0][2]-b.xyxy[0][0])*(b.xyxy[0][3]-b.xyxy[0][1])
            ↪ for b in cls_boxes)
            features[f'{cls_name}_max'] = max([(b.xyxy[0][2]-b.xyxy[0][0])*(b.xyxy[0][3]-b.xyxy[0][1])
            ↪ for b in cls_boxes]) if cls_boxes else 0

        features['total_defect_ratio'] = sum(features.get(f'{c}_area',0) for c in
        ↪ ['wuzi','waterstain','yellowstain','nodule','airhole','bubble','bump','fawu','overfire','NiOH','greenstain',
        ↪ / total_area
        features['has_nodule'] = 1 if any(int(b.cls[0]) == 3 for b in boxes) else 0

        records.append(features)

    pd.DataFrame(records).to_csv(output_csv, index=False)
    print(f"Features saved: {output_csv}")

if __name__ == '__main__':
    extract_features(
        image_dir='datasets/dianjienie/images/train/single',
        model_path='runs/dianjienie/phase2/weights/best.pt',
        output_csv='datasets/dianjienie/grade_labels/features.csv'
    )
```

5.2 LightGBM 训练

```
# scripts/train_lgbm.py
import pandas as pd
import lightgbm as lgb
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
import joblib

def train():
    # 合并特征和等级标签
    features = pd.read_csv('datasets/dianjienie/grade_labels/features.csv')
    grades = pd.read_csv('datasets/dianjienie/grade_labels/old_grades.csv')
    df = features.merge(grades, on='image_id')

    X = df.drop(['image_id', 'grade_code'], axis=1)
    y = df['grade_code']

    X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2, random_state=42)

    train_data = lgb.Dataset(X_train, label=y_train)
    val_data = lgb.Dataset(X_val, label=y_val, reference=train_data)

    params = {
        'objective': 'multiclass',
```

```
        'num_class': len(y.unique()),
        'metric': 'multi_error',
        'learning_rate': 0.05,
        'num_leaves': 31,
        'verbose': -1
    }

    model = lgb.train(params, train_data, num_boost_round=200,
                      valid_sets=[val_data],
                      callbacks=[lgb.early_stopping(20)])

    y_pred = model.predict(X_val)
    acc = accuracy_score(y_val, y_pred.argmax(axis=1))
    print(f"Accuracy: {acc:.4f}")

    joblib.dump(model, 'models/dianjienie_grade.pkl')
    joblib.dump(list(X.columns), 'models/dianjienie_grade_features.pkl')
    print("Model saved.")

if __name__ == '__main__':
    train()
```

六、部署：Tauri + FastAPI Sidecar

6.1 架构

```
Tauri App（用户双击打开）
- Frontend: React/Vue
- Sidecar: Python Backend（PyInstaller 打包的 exe）
  - FastAPI 服务（localhost:5005）
  - ONNX Runtime（加载 YOLO11 ONNX）
  - LightGBM（加载等级模型）
- 关闭 Tauri 时自动杀死 Sidecar
```

6.2 FastAPI 后端

```
# backend/main.py
from fastapi import FastAPI, File, UploadFile
from fastapi.responses import JSONResponse
import numpy as np
import cv2
import onnxruntime as ort
import joblib
import uvicorn

app = FastAPI()

# 加载模型（启动时）
detector = ort.InferenceSession('models/dianjienie_yolo11.onnx')
```

```
grade_model = joblib.load('models/dianjienie_grade.pkl')
feature_names = joblib.load('models/dianjienie_grade_features.pkl')

@app.post("/detect")
async def detect(image: UploadFile = File(...)):
    contents = await image.read()
    nparr = np.frombuffer(contents, np.uint8)
    img = cv2.imdecode(nparr, cv2.IMREAD_COLOR)

    # 预处理
    h, w = img.shape[:2]
    scale = min(640/h, 640/w)
    new_h, new_w = int(h*scale), int(w*scale)
    resized = cv2.resize(img, (new_w, new_h))
    padded = np.full((640,640,3), 114, dtype=np.uint8)
    padded[:new_h, :new_w] = resized
    input_tensor = np.transpose(padded.astype(np.float32)/255.0, (2,0,1))
    input_tensor = np.expand_dims(input_tensor, axis=0)

    # 检测
    outputs = detector.run(None, {detector.get_inputs()[0].name: input_tensor})

    # 后处理（简化版，实际需要完整 NMS）
    # ... 解析 outputs，提取缺陷列表 ...
    defects = []

    # 等级判定
    features = {name: 0 for name in feature_names}
    # ... 填充缺陷统计 ...
    x = np.array([[features.get(name, 0) for name in feature_names]])
    proba = grade_model.predict(x)[0]
    grade = int(proba.argmax())

    return JSONResponse({
        'defects': defects,
        'grade': grade,
        'confidence': float(proba[grade])
    })

@app.get("/health")
async def health():
    return {"status": "ok"}

if __name__ == "__main__":
    uvicorn.run(app, host="127.0.0.1", port=5005)
```

6.3 PyInstaller 打包

```
# 安装
pip install pyinstaller
```

```
# 打包（包含模型文件）
pyinstaller --onefile --name nickel_backend     --add-data "models/dianjienie_yolo11.onnx:models"
↪  --add-data "models/dianjienie_grade.pkl:models"     --add-data "models/
↪  dianjienie_grade_features.pkl:models"     --hidden-import onnxruntime     --hidden-import lightgbm
↪  backend/main.py

# 输出: dist/nickel_backend.exe
```

6.4 Tauri 配置（启动 Sidecar）

```
// src-tauri/tauri.conf.json
{
  "tauri": {
    "allowlist": {
      "shell": {
        "sidecar": true,
        "scope": [{"name": "binaries/nickel_backend", "sidecar": true}]
      },
      "http": {"all": true, "request": true}
    },
    "bundle": {
      "active": true,
      "targets": "all",
      "identifier": "com.nickel.defect"
    }
  }
}
```

```
// src-tauri/src/main.rs
use tauri::api::process::{Command, CommandEvent};

fn main() {
    tauri::Builder::default()
        .setup(|app| {
            let sidecar = Command::new_sidecar("nickel_backend")
                .expect("Failed to create sidecar");
            let (mut rx, _child) = sidecar.spawn().expect("Failed to spawn");

            tauri::async_runtime::spawn(async move {
                while let Some(event) = rx.recv().await {
                    if let CommandEvent::Stdout(line) = event {
                        println!("Backend: {}", line);
                    }
                }
            });
            Ok(())
        })
        .run(tauri::generate_context!())
        .expect("Error");
}
```

6.5 Tauri 前端调用

```tsx
// src/App.tsx
import { useState } from 'react';
import { fetch } from '@tauri-apps/api/http';

function App() {
  const [result, setResult] = useState<any>(null);

  const detect = async (file: File) => {
    const form = new FormData();
    form.append('image', file);

    const res = await fetch('http://127.0.0.1:5005/detect', {
      method: 'POST',
      body: form as any,
    });

    setResult(res.data);
  };

  return (
    <div>
      <h1> 镍板缺陷检测 </h1>
      <input type="file" accept="image/*" onChange={(e) => e.target.files && detect(e.target.files[0])}
      ↪ />
      {result && (
        <div>
          <p> 等级: {result.grade}</p>
          <p> 缺陷数: {result.defects.length}</p>
        </div>
      )}
    </div>
  );
}
```

七、执行清单（按顺序）

Week 1：数据

- 整理旧数据集（单光图片 + YOLO 标签）
- 收集新多光源图片
- 运行 preprocess.py 生成伪彩色
- 人工审核标签（重点：中低置信度）

Week 2-3：训练检测模型

- Phase 1: 冻结 Backbone 训练（100 epochs）
- Phase 2: 全量微调（50 epochs）
- 验证精度（mAP@0.5 >= 80%）

- 导出 ONNX

Week 4：训练等级模型

- 用检测模型跑旧数据，提取特征
- 训练 LightGBM
- 验证准确率（>= 90%）

Week 5：后端

- 开发 FastAPI 服务
- PyInstaller 打包成 exe
- 测试本地 API

Week 6：前端

- Tauri + React 项目初始化
- 开发图片上传和结果显示界面
- 集成 Sidecar 启动后端
- 测试完整流程

Week 7：测试发布

- 产线测试
- 与人工判定对比
- 调整阈值（改 YAML 配置）
- 打包发布

版本：v2.0
面向：单人全栈开发者
最后更新：2026-07-06

Kimi.ai 生成